

Memoria RESONUM

Método de Newton para la inversa matricial

Autores:

Melgarejo Riquelme, Vicente

Aroca Martínez, Unai

Zaballa Martínez, Alberto



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Grado de Matemáticas

Universidad Politécnica de Valencia

Índice

1. Planteamiento del método	2
2. Desarrollo, Convergencia y Resultados	2
2.1. Deducción de la expresión iterativa	2
2.2. Acotación del residuo E_k por inducción	3
2.3. Convergencia del residuo a cero	3
2.4. Orden de convergencia	3
2.5. Implementación numérica y resultados	4
3. Estudio dinámico del método de Newton	8
3.1. Obtención del operador	8
3.2. Estudio de los puntos fijos	8
3.3. Puntos críticos	9
3.4. Representación del plano dinámico	9
4. Inversa de Moore-Penrose	11
5. Modificaciones del método	14
5.1. Transformación del método de Traub para $F(X) = 0$	14
5.2. Método multipaso congelando la evaluación	15

1. Planteamiento del método

Sea $A \in \mathbb{C}^{n \times n}$ una matriz invertible. Consideramos la ecuación matricial no lineal $F(X) = X^{-1} - A = 0$, cuya solución exacta es la matriz inversa de A .

El método de Newton aplicado para aproximar la solución de $F(X) = 0$ da lugar a la siguiente expresión iterativa:

$$X_{k+1} = X_k(2I - AX_k), \quad k = 0, 1, \dots,$$

donde I denota la matriz identidad.

2. Desarrollo, Convergencia y Resultados

2.1. Deducción de la expresión iterativa

Esta expresión iterativa se deduce de aplicar el método de Newton para funciones matriciales:

$$X_{k+1} = X_k - [F'(X_k)]^{-1} F(X_k)$$

El principal problema en esta expresión es la $[F'(X_k)]^{-1}$, al ser la "derivada" de una función matricial, su forma es incierta, la definiremos directamente con el siguiente límite:

$$\begin{aligned} F'(X) &:= \\ &= \lim_{h \rightarrow 0} \frac{F(X+h) - F(X)}{h} = \lim_{h \rightarrow 0} \frac{((X+h)^{-1} - A) - (X^{-1} - A)}{h} = \lim_{h \rightarrow 0} \frac{(X+h)^{-1} - X^{-1}}{h} = \\ &= \lim_{h \rightarrow 0} \frac{(X+h)^{-1} (I - (X+h)X^{-1})}{h} = \lim_{h \rightarrow 0} \frac{(X+h)^{-1} (X - (X+h)) X^{-1}}{h} = \\ &= \lim_{h \rightarrow 0} \frac{(X+h)^{-1} (-hI) X^{-1}}{h} = \lim_{h \rightarrow 0} (X+h)^{-1} (-X)^{-1} = \\ &= \lim_{h \rightarrow 0} (-X^2 - hX)^{-1} = (-X^2)^{-1} = -X^{-2} \end{aligned}$$

Con esto ya podemos sustituir en nuestra expresión iterativa:

$$\begin{aligned} X_{k+1} &= X_k - [F'(X_k)]^{-1} F(X_k) = X_k - [-X_k^{-2}]^{-1} * [X_k^{-1} - A] \\ &= X_k + X_k^2 * [X_k^{-1} - A] = X_k + X_k - X_k^2 * A = X_k * [2I - X_k * A] \end{aligned}$$

Con esta forma de tratar $F'(X_k)^{-1}$ no llegamos exactamente a la expresión que buscábamos, $X_{k+1} = X_k(2I - AX_k)$, por ello, buscaremos desarrollar el método de forma escalar, considerando la función escalar análoga:

$$\begin{aligned} f(x) &= \frac{1}{x} - a, \quad \forall a \in \mathbb{R} \setminus \{0\} \\ f'(x) &= -\frac{1}{x^2} \end{aligned}$$

Desarrollamos el método de Newton:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{\frac{1}{x_k} - a}{-\frac{1}{x_k^2}} = x_k + x_k^2 \frac{1 - ax_k}{x_k} = x_k + x_k(1 - ax_k) = x_k(2 - ax_k)$$

Extrapolando esta expresión a matrices, se deduce lo que buscábamos:

$$X_{k+1} = X_k(2I - AX_k)$$

2.2. Acotación del residuo E_k por inducción

Sabiendo que la solución exacta de nuestra ecuación $F(X) = 0$ es la inversa de A , podemos definir la matriz residuo en el paso k como:

$$E_k = I - AX_k$$

Queremos demostrar por inducción que $\|E_k\| < 1, \forall k$ para todas las iteraciones, si comenzamos con una aproximación inicial que cumpla $\|I - AX_0\| < 1$.

Paso base ($k = 0$): Por hipótesis, sabemos que $\|E_0\| < 1$.

Paso inductivo: Suponemos cierto para k y demostramos para $k + 1$. Comenzamos desarrollando a partir de E_{k+1} , buscando E_k :

$$E_{k+1} = I - AX_{k+1} = I - A[X_k(2I - AX_k)] = I - 2AX_k + AX_k * AX_k = I - AX_k - AX_k + (AX_k)^2 =$$

Sacamos factor común $-AX_k$ y luego $(I - AX_k)$:

$$= I - AX_k - AX_k(I - AX_k) = (I - AX_k)(I - AX_k) = E_k^2$$

Por tanto, teniendo en cuenta la hipótesis de inducción $\|E_k\| < 1$:

$$\|E_{k+1}\| = \|E_k^2\| = \|E_k\|^2 < 1^2 = 1$$

2.3. Convergencia del residuo a cero

Queremos garantizar que el método converge a A^{-1} cuándo se cumple nuestra hipótesis $\|I - AX_0\| < 1$. Para ello, estudiamos si el residuo converge a 0, ya que esto implicaría que el producto AX_k tiende a I .

Del apartado anterior, tenemos que $E_{k+1} = E_k^2, \forall k$, repitiendo este razonamiento:

$$E_k = (E_{k-1})^2 = (E_{k-2}^2)^2 = E_{k-2}^4 = \dots = E_1^{2^{k-1}} = E_0^{2^k}$$

Dado que $\|E_0\| < 1$, podemos tomar límites de la forma siguiente:

$$\lim_{k \rightarrow \infty} \|E_k\| = \lim_{k \rightarrow \infty} \|E_0^{2^k}\| = \lim_{k \rightarrow \infty} \|E_0\|^{2^k} = 0$$

Con esto, concluimos que, con una aproximación inicial correcta, nuestro método es convergente y converge a la solución de $F(X) = 0$, que es A^{-1} . Se puede apreciar también que, de no cumplir $\|E_0\|$ la condición, el método divergirá, con lo cuál esta es una condición necesaria.

2.4. Orden de convergencia

Queremos determinar el orden de convergencia de este método iterativo. Sea el error $e_k = X_k - A^{-1}$. Vamos a demostrar que obtenemos una relación del tipo: $\|e_{k+1}\| \leq C \|e_k\|^2$, donde C es una constante positiva.

Para obtener la expresión del error, podemos partir directamente de la expresión iterativa de Newton y modificarla para demostrar que se obtiene convergencia cuadrática.

$$X_{k+1} = 2X_k - X_k AX_k$$

$$X_{k+1} - A^{-1} = 2X_k - X_k AX_k - A^{-1}$$

Ponemos $X_{k+1} - A^{-1}$ como e_{k+1} y separamos $2X_k$ en dos sumandos:

$$\begin{aligned}
e_{k+1} &= X_k + X_k - X_k A X_k - A^{-1} \\
&= X_k - A^{-1} + X_k - X_k A X_k \\
&= e_k + X_k - X_k A X_k \\
&= e_k + X_k(I - A X_k)
\end{aligned}$$

Ponemos I como el producto de AA^{-1} para poder sacar factor común:

$$\begin{aligned}
&= e_k + X_k(AA^{-1} - A X_k) \\
&= e_k + X_k A(A^{-1} - X_k) \\
&= e_k + X_k A(-e_k) \\
&= e_k - X_k A e_k \\
&= e_k(I - X_k A)
\end{aligned}$$

Ponemos I como el producto de $A^{-1}A$ para poder sacar factor común:

$$\begin{aligned}
&= e_k(A^{-1}A - X_k A) \\
&= e_k(A^{-1} - X_k)A \\
&= e_k(-e_k)A \\
&= -e_k^2 A
\end{aligned}$$

Ahora tomamos normas a ambos lados y aplicamos desigualdades:

$$\begin{aligned}
\|e_{k+1}\| &= \|-e_k^2 A\| \\
&\leq \|e_k\|^2 \|A\| \\
&\leq C \|e_k\|^2
\end{aligned}$$

Por lo tanto, queda demostrada la convergencia cuadrática (con $C = \|A\|$).

2.5. Implementación numérica y resultados

Finalmente, se ha implementado el algoritmo para distintas matrices random de tamaños $n = 3, 10, 50, 100$ y distintas aproximaciones iniciales X_0 , todas ellas de la forma $X_0 = \beta A^t$. Para ello fijamos una semilla arbitraria, en este caso la 33, con el fin de conseguir que todas las matrices con las que probamos los distintos valores de beta sean las mismas para poder comparar los resultados. Todo ellos se ilustra en la siguiente tabla, donde hemos utilizado como criterio de parada que $\|I - AX_0\| < tol$, a lo que hemos llamado Incre. En nuestro caso, la tol que hemos utilizado es de $1e-10$.

n	β	Iter	$\ X_{k+1} - X_k\ $	Incre	ACOC	TCC	TCL	Time
3	$\frac{1}{\ A\ ^2}$	14	$1,8 \times 10^{-10}$	$7,5 \times 10^{-12}$	1.8137	$1,8 \times 10^3$	$1,9 \times 10^{-6}$	0.01406
	$\frac{1}{2\ A\ ^2}$	15	$1,8 \times 10^{-10}$	$7,7 \times 10^{-12}$	1.8192	$1,9 \times 10^3$	$2,0 \times 10^{-9}$	0.02031
	$\frac{1}{4\ A\ ^2}$	16	$1,8 \times 10^{-10}$	$7,8 \times 10^{-12}$	1.8169	$2,6 \times 10^3$	$2,1 \times 10^{-6}$	0.01906
	$\frac{1}{8\ A\ ^2}$	17	$1,9 \times 10^{-10}$	$7,9 \times 10^{-12}$	1.8226	$1,2 \times 10^3$	$1,8 \times 10^{-6}$	0.02156
	$\frac{1}{16\ A\ ^2}$	16	$1,9 \times 10^{-10}$	$7,9 \times 10^{-12}$	1.8226	$1,2 \times 10^3$	$1,8 \times 10^{-6}$	0.02156
10	$\frac{1}{\ A\ ^2}$	20	$3,9 \times 10^{-10}$	$3,6 \times 10^{-12}$	1.7988	$6,7 \times 10^3$	$2,7 \times 10^{-6}$	0.01875
	$\frac{1}{2\ A\ ^2}$	21	$3,9 \times 10^{-10}$	$3,6 \times 10^{-12}$	1.7984	$5,4 \times 10^3$	$2,5 \times 10^{-6}$	0.01750
	$\frac{1}{4\ A\ ^2}$	22	$3,9 \times 10^{-10}$	$3,6 \times 10^{-12}$	1.8004	$6,3 \times 10^3$	$2,7 \times 10^{-6}$	0.01343
	$\frac{1}{8\ A\ ^2}$	23	$3,9 \times 10^{-10}$	$3,6 \times 10^{-12}$	1.7985	$5,4 \times 10^3$	$2,5 \times 10^{-6}$	0.02250
	$\frac{1}{16\ A\ ^2}$	24	$3,9 \times 10^{-10}$	$3,6 \times 10^{-12}$	1.8026	$5,0 \times 10^3$	$2,4 \times 10^{-6}$	0.03062
50	$\frac{1}{\ A\ ^2}$	28	$9,8 \times 10^{-10}$	$7,5 \times 10^{-12}$	1.5337	$1,7 \times 10^6$	$4,3 \times 10^{-4}$	0.19062
	$\frac{1}{2\ A\ ^2}$	29	$9,0 \times 10^{-10}$	$7,5 \times 10^{-12}$	1.5353	$1,6 \times 10^6$	$4,0 \times 10^{-4}$	0.14250
	$\frac{1}{4\ A\ ^2}$	30	$1,0 \times 10^{-9}$	$7,6 \times 10^{-12}$	1.5310	$1,5 \times 10^6$	$3,8 \times 10^{-4}$	0.14750
	$\frac{1}{8\ A\ ^2}$	31	$9,6 \times 10^{-10}$	$7,5 \times 10^{-12}$	1.5355	$1,5 \times 10^6$	$3,7 \times 10^{-4}$	0.18000
	$\frac{1}{16\ A\ ^2}$	32	$9,9 \times 10^{-10}$	$7,5 \times 10^{-12}$	1.5310	$1,6 \times 10^6$	$4,1 \times 10^{-4}$	0.18093
100	$\frac{1}{\ A\ ^2}$	29	$9,0 \times 10^{-10}$	$1,1 \times 10^{-11}$	1.5275	$7,1 \times 10^5$	$1,4 \times 10^{-4}$	0.48468
	$\frac{1}{2\ A\ ^2}$	30	$9,1 \times 10^{-10}$	$1,1 \times 10^{-11}$	1.5257	$7,1 \times 10^5$	$1,5 \times 10^{-4}$	0.45625
	$\frac{1}{4\ A\ ^2}$	31	$9,0 \times 10^{-10}$	$1,1 \times 10^{-11}$	1.5248	$7,1 \times 10^5$	$1,4 \times 10^{-4}$	0.47935
	$\frac{1}{8\ A\ ^2}$	32	$9,0 \times 10^{-10}$	$1,1 \times 10^{-11}$	1.5373	$6,6 \times 10^5$	$1,3 \times 10^{-4}$	0.48875
	$\frac{1}{16\ A\ ^2}$	33	$9,1 \times 10^{-10}$	$1,1 \times 10^{-11}$	1.5275	$7,7 \times 10^5$	$1,5 \times 10^{-4}$	0.47850

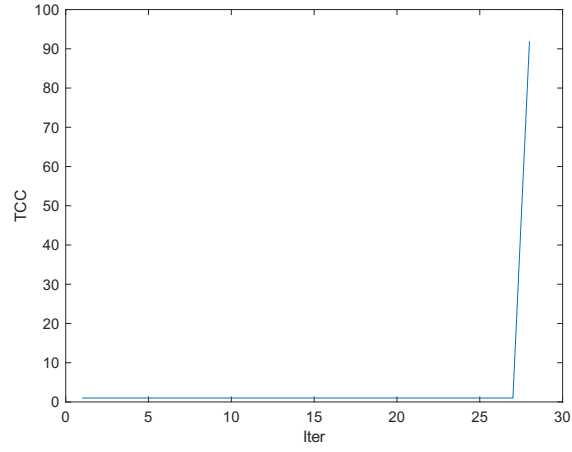


Figura 1: TCC para $n = 100$ y $\beta = \frac{1}{\|A\|^2}$

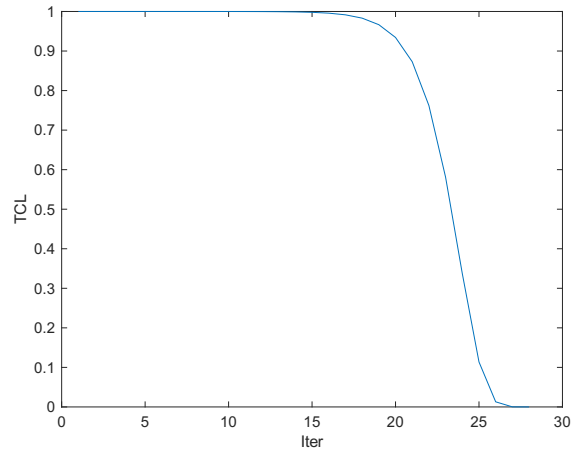


Figura 2: TCL para $n = 100$ y $\beta = \frac{1}{\|A\|^2}$

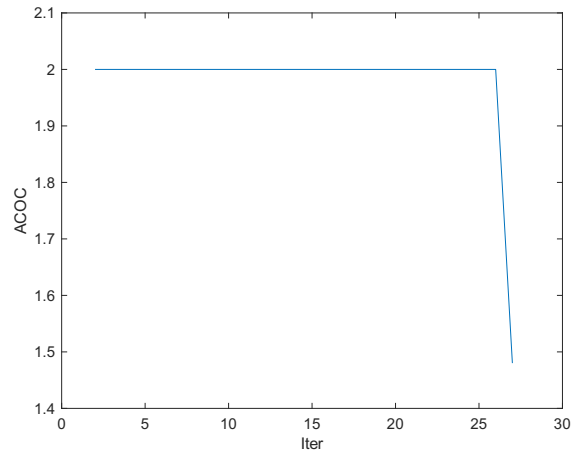


Figura 3: ACOC para $n = 100$ y $\beta = \frac{1}{\|A\|^2}$

En relación con los datos de la tabla podemos observar varias cosas, la primera es que los valores de beta que hemos elegido son valores pequeño, con el fin de que nuestra condición necesaria, es decir, $\|I - AX_0\| < 1$, se cumpla. Por otro lado vemos que las iteraciones aumentan a medida que aumenta el tamaño de las matrices aunque no de forma desmesurada. En relación con el ACOC y las tasas de convergencia observamos que en este caso en ACOC no proporciona información relevante, pues como podemos observar en las columnas de TCC (tasa de convergencia cuadrática) y TCL (tasa de convergencia lineal), el método tiene orden de convergencia cuadrático, pues la tasa lineal tiende a cero y la cuadrática se estabiliza, en cambio el ACOC en la mayoría de casos está en torno a 1.5. Por último, lo que también podemos apreciar es que el tiempo de ejecución aumenta considerablemente a medida que aumentamos los tamaños a diferencia del numero de iteraciones que como hemos mencionado anteriormente aumenta pero no significativamente.

A continuación se muestra el método iterativo programado en MATLAB

```
function [iter,SOL,incre,I,tcl,tcc,incre1,ACOC,dif_iter] = Newton_matrcial(A,X0,maxiter,tol)
%[iter,SOL,incre,I,tcl,tcc,incre1,ACOC,dif_iter] = Newton_matrcial(A,X0,maxiter,tol)
n = size(A); n = n(1);
incre1 = norm(eye(n)-A*X0);
I = []; %vector de incrementos.
I = [I,incre1];
iter = 0;
incre = incre1;
dif_iter = []; %diferencia entre dos iterados consecutivos
%Utilizmaos como incremento I-A*X0 pues la sol es la inversa de A
if (incre<1)
    while(incre>tol && iter<maxiter)
        X = X0*(2*eye(n)-A*X0);
        incre = norm(eye(n)-A*X0);
        Dif = norm(X-X0);
        X0 = X;
        iter = iter+1;
        I = [I,incre];
        dif_iter = [dif_iter,Dif];
    end

    if (iter >= maxiter)
        disp('Se ha superado el número máximo de iteraciones')
        SOL=[];
    else
        SOL = X;

    end
    figure;
    ACOC = log(I(3:end)./I(2:end-1))./log(I(2:end-1)./I(1:end-2));
    tcl=I(2:end)./I(1:end-1);subplot(3,1,1);plot(tcl);legend('Tasa lineal')
    tcc=I(2:end)./(I(1:end-1).^2);subplot(3,1,2);plot(tcc);legend('Tasa cuadrática')
    subplot(3,1,3);
    plot(ACOC);
    legend('ACOC')
else
    disp('No se cumple la condición para la norma del error')
    SOL=[];
end
```

3. Estudio dinámico del método de Newton

En esta sección estudiaremos la dinámica del método de Newton aplicado a la siguiente función:

$$f(x) = \frac{1}{x} - a, a \in \mathbb{R} \setminus \{0\}$$

Se puede apreciar que esta es la función escalar análoga a la trabajada matricialmente en los apartados anteriores, por lo cuál su solución es también la inversa de a . Al ser la solución de la ecuación $f(x) = 0$ la inversa de a , es correcto asumir $a \neq 0$, análogo a que A era invertible en el caso anterior.

3.1. Obtención del operador

Para obtener el operador de newton asociado a esta función, ejecutamos el siguiente código en Mathematica:

```
f[x_] = 1/x - a;  
Nw[x_] = Simplify[x - f[x]/f'[x]]
```

Obtenemos el siguiente operador:

$$Nw[x] = -x(-2 + ax)$$

3.2. Estudio de los puntos fijos

Para estudiar la convergencia del método, buscamos los puntos fijos resolviendo $Nw[x] = x$, con el siguiente código:

```
Solve[Nw[x] == x, x]
```

Con lo que obtenemos que hay dos puntos fijos, $x = 0$ y $x = \frac{1}{a}$. Siendo el primero un punto fijo extraño y el segundo la solución de nuestra ecuación $f(x) = 0$.

Para estudiar la estabilidad del método respecto a estos puntos fijos, obtenemos el valor de la derivada:

```
dOpN[x_] = Nw'[x]; // Simplify  
dOpN[0]  
dOpN[1/a]
```

Con esto obtenemos que $dOpN[0] = 2$, por lo cuál es un punto fijo repulsor, tal como necesitamos, al ser un valor distinto a nuestra solución. Por otro lado obtenemos $dOpN[1/a] = 0$, por lo que es un punto fijo superatractor, lo que es la situación ideal.

Debido a la forma específica de nuestra función (considerada en la esfera de Riemann) y a que luego encontraremos dos cuencas de atracción, estudiaremos cómo se comporta nuestro operador respecto al infinito, a pesar de no haber aplicado Möbius.

```
invOp[x_] = Simplify[1/Nw[1/x]]  
invOp[0]  
invOp'[0]
```

Al ejecutar esto, ambas evaluaciones son 0, con lo que el infinito es un punto fijo, además, es superatractor, lo que se verá luego reflejado en el plano dinámico.

3.3. Puntos críticos

Para obtener los puntos críticos, igualamos la derivada a 0.

```
Solve[dOpN[x] == 0, x]
```

El único punto crítico que obtenemos así es $\frac{1}{a}$, que además es la solución de la ecuación. Sin embargo, esta evaluación excluye al infinito que, como vimos antes al ejecutar `invOp'[0]` y obtener 0, también es un punto crítico. Como no hay críticos libres, no habrán cuencas de atracción además de la de $\frac{1}{a}$, la solución y la de ∞ , la 'divergencia'.

3.4. Representación del plano dinámico

Buscamos representar el plano dinámico para ver a simple vista qué estimaciones iniciales convergen a la solución y cuáles divergen. Utilizaremos el programa `planodinam.m`, modificando el operador por el de nuestro caso particular, `Op=-x*(-2+aa*x);`.

Ejecutamos el comando `[I,it]=planodinam(a,[-5 5 -5 5],400,40);` con diferentes valores de a , y vemos las distintas cuencas de convergencia.

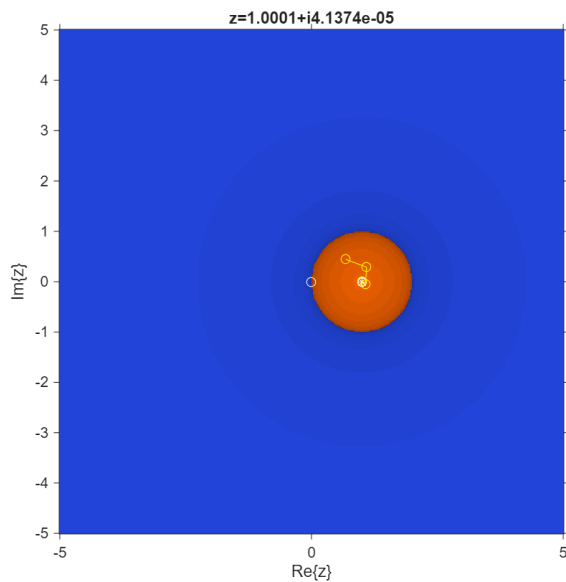


Figura 4: $a = 1$

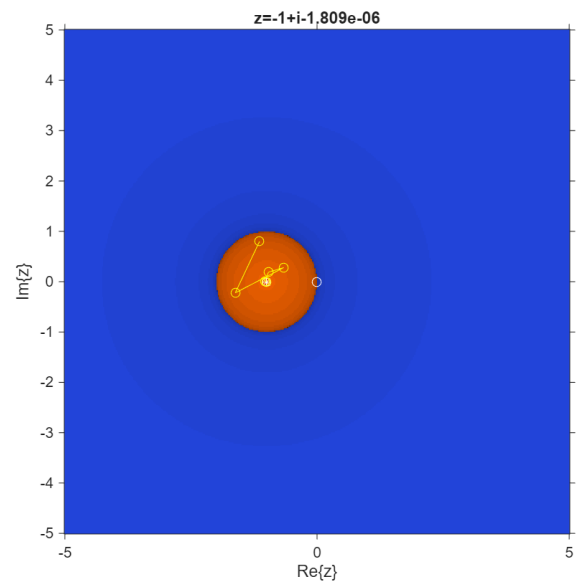


Figura 5: $a = -1$

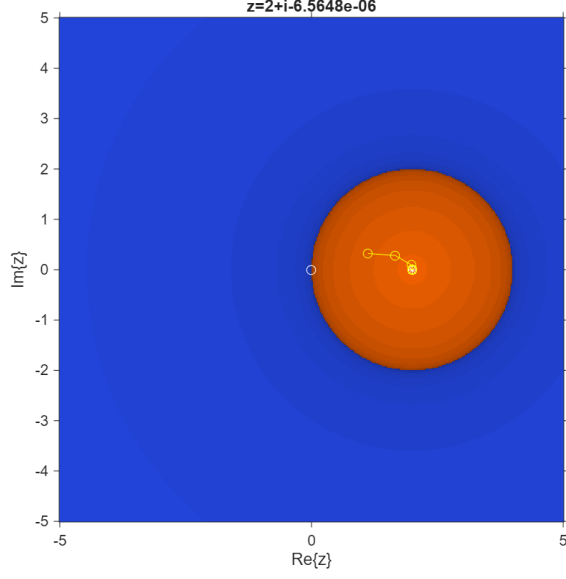


Figura 6: $a = 0,5$

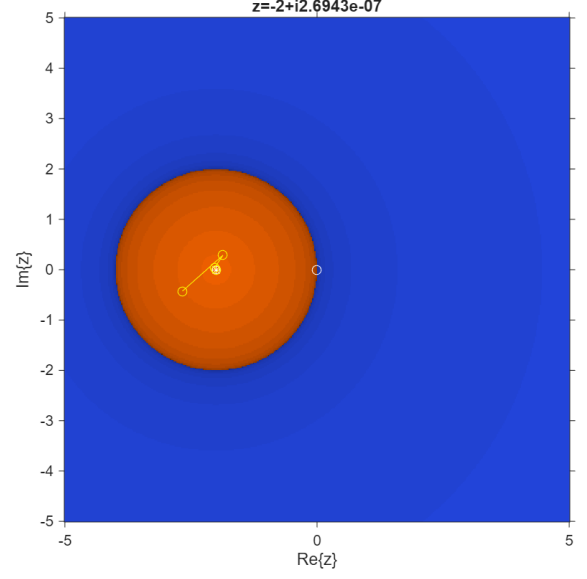


Figura 7: $a = -0,5$

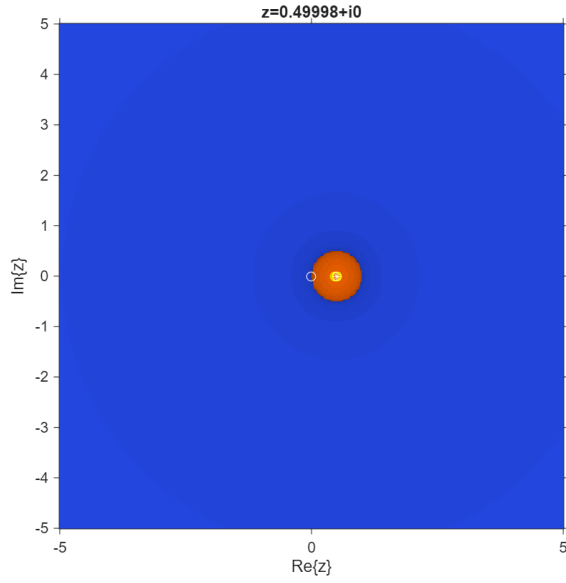


Figura 8: $a = 2$

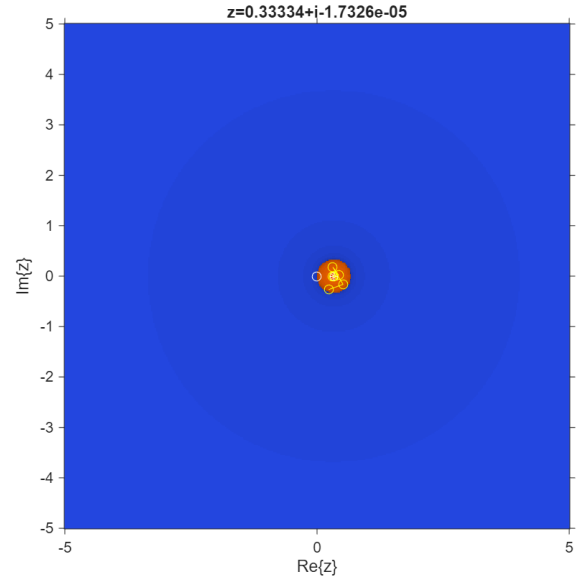


Figura 9: $a = 3$

En todas las imágenes podemos ver que la cuenca roja de la solución $\frac{1}{a}$ es un disco centrado en $\frac{1}{a}$ y de radio $|\frac{1}{a}|$. Cualquier estimación inicial dentro del círculo, hará que converja a la solución, mientras que cualquiera fuera, en la cuenca azul, hará que el método 'diverja' o converja al infinito.

Es pertinente apreciar que esta condición se puede escribir, siendo x_0 la aproximación inicial, de la siguiente manera:

$$\left| \frac{1}{a} - x_0 \right| < \left| \frac{1}{a} \right| \implies |1 - ax_0| < 1$$

Que, otra vez, es una condición de convergencia análoga al caso matricial, y al igual que en ese caso, es una condición necesaria.

4. Inversa de Moore-Penrose

Queremos saber si nuestro método se puede aplicar a matrices no cuadradas $A \in \mathbb{K}^{n \times m}$, como estas matrices no son invertibles, entonces no se puede aproximar su inversa, si no que vamos a aproximar la pseudoinversa o inversa de Moore-Penrose. La respuesta a esta pregunta es que sí se puede hacer con nuestro método, esto tiene una demostración analítica que dejaremos en la bibliografía. (1)

Para ello hemos modificado ligeramente el programa realizado en la cuestión 1, el cambio principal es el criterio de parada, ahora usaremos el siguiente:

$$\|AX_{k+1}A - A\| + \|(AX_{k+1})^t - AX_{k+1}\| + \|X_{k+1}AX_{k+1} - X_{k+1}\| + \|(X_{k+1}A)^t - X_{k+1}A\| < tol$$

Es decir, estamos exigiendo que se cumplan todas las propiedades de dicha inversa haciendo que la suma del las normas del error en la iteración k+1 sea menor que una cierta tolerancia, y ha esta suma en cada iteración es a lo que llamaremos incremento, de forma abreviada incre.

Para ver y analizar el funcionamiento de nuestro programa vamos a realizar una tabla en la que se indicará el tamaño de las matrices, donde n son las filas y m las columnas, el numero de iteraciones, ACOC, tasas de convergencia lineal y cuadrática, el incre y la diferencia entre dos iterados consecutivos así como el tiempo medio de ejecución. Al igual que en la cuestión 1, se ha usado la semilla arbitraria 33, y para cada número de filas y columnas se han generado 50 matrices random, la media de los resultados obtenidos en nuestro programa es lo que aparece en la tabla. Para la realización de esta tabla hemos utilizado una $tol = 1e^{-10}$. La estimación inicial es $X_0 = \beta A^t$, donde $\beta = \frac{1}{\|A\|^2}$. Cabe destacar que para la obtención de estas matrices nos aseguramos previamente de que $\|E_k\| < 1$ pues es una condición necesaria para la convergencia como hemos visto en el pimer ejercicio.

n	m	Iter	$\ X_{k+1} - X_k\ $	Incre	ACOC	TCC	TCL	Time
3	2	11	1.3×10^{-6}	9.1×10^{-12}	1.6377	7.9×10^3	2.4×10^{-6}	0.13187
2	3	10	1.0×10^{-6}	4.3×10^{-12}	1.6976	4.4×10^3	2.2×10^{-6}	0.06843
10	5	12	8.2×10^{-7}	2.2×10^{-12}	1.6172	1.2×10^3	3.0×10^{-6}	0.08218
5	10	12	2.0×10^{-6}	8.4×10^{-12}	1.7200	4.3×10^3	2.1×10^{-6}	0.08437
50	25	15	1.4×10^{-6}	6.5×10^{-12}	1.7213	1.4×10^4	6.6×10^{-6}	0.92500
25	50	15	1.2×10^{-6}	6.0×10^{-12}	1.6990	1.8×10^4	7.8×10^{-6}	0.93125
75	50	18	7.9×10^{-6}	3.6×10^{-12}	1.3742	1.4×10^4	1.4×10^{-7}	0.27772
50	75	18	1.3×10^{-6}	6.7×10^{-12}	1.5403	3.27×10^5	4.7×10^{-5}	0.28031

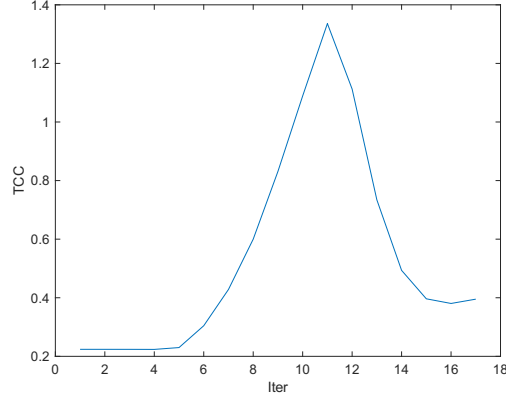


Figura 10: TCC para $n = 50$, $m = 75$ y $\beta = \frac{1}{\|A\|^2}$

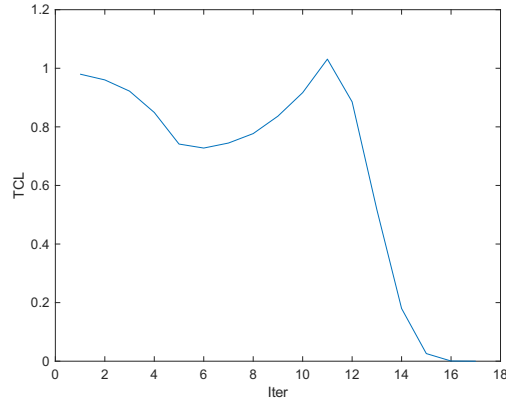


Figura 11: TCL para $n = 50$, $m = 75$ y $\beta = \frac{1}{\|A\|^2}$

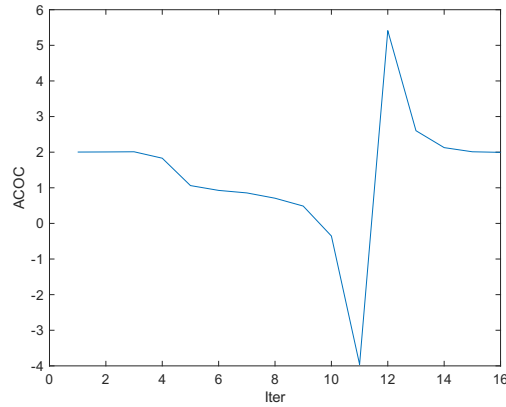


Figura 12: ACOC para $n = 50$, $m = 75$ y $\beta = \frac{1}{\|A\|^2}$

En la tabla se observa que tiene una convergencia cuadrática, ya que la tasa lineal tiende a cero y la cuadrática se estabiliza, además el ACOC está cercano a dos. Por otro lado, vemos que no le hace falta un número muy alto de iteraciones y que el tiempo medio de ejecución del programa aumenta a medida que aumentamos el tamaño. Además, podemos ver que independientemente de si $n > m$ o $n < m$ se obtienen resultado similares. Podemos ver que las gráficas concuerdan con los resultados obtenidos en la tabla. Es

interesante ver que el ACOC aquí aporta muy poca información pues vemos que no para de oscilar, aunque en las últimas iteraciones si que va tendiendo a la convergencia cuadrática que es la que tiene el método.

A continuación se muestra el código de MATLAB empleado:

```
function [SOL, iter, I, incre, tcl,tcc,ACOC,dif_iter] = Newton_No_Cuadrado(A,X0,maxiter,tol)
%SOL, iter, I, incre, tcl,tcc,ACOC,dif_iter] = Newton_NO_Cuadrado(A,X0, maxiter,tol)
t = size(X0);
n = t(2);

incre1(1) = norm(A*X0*A-A);
incre1(2) = norm((A*X0)'-A*X0);
incre1(3) = norm(X0*A*X0-X0);
incre1(4) = norm((X0*A)'-X0*A);
incre = sum(incre1);
iter = 1;
I = []; I=[I,incre];
dif_iter = [];
while(iter<maxiter && incre > tol)
    XK = X0*(2*eye(n)-A*X0);
    dif = norm(XK-X0);
    incre1(1) = norm(A*XK*A-A);
    incre1(2) = norm((A*XK)'-A*XK);
    incre1(3) = norm(XK*A*XK-XK);
    incre1(4) = norm((XK*A)'-XK*A);
    incre = sum(incre1);
    iter = iter+1;
    X0 = XK;
    I=[I,incre];
    dif_iter = [dif_iter,dif];
end
if iter >= maxiter
    disp('Se ha superado el numero maximo de iteraciones')
    SOL = [];
else
    disp('Se ha encontrado la solución')
    SOL = XK;
end
figure;
ACOC = log(I(3:end)./I(2:end-1))./log(I(2:end-1)./I(1:end-2));
tcl=I(2:end)./I(1:end-1);
tcc=I(2:end)./(I(1:end-1).^2);
end
```

Se observan algunas modificaciones en el código respecto del anterior, ya que ahora calculamos varios incrementos, los cuales sirven para garantizar que se cumplen todas las condiciones de la inversa de Moore-Penrose.

5. Modificaciones del método

5.1. Transformación del método de Traub para $F(X) = 0$

Vamos a trabajar con el método de Traub, para ver si conseguimos quitarnos la inversa, ya que aumenta mucho el coste computacional, además que encontrar la inversa es el propio objetivo del método. Vamos a trabajar mediante elementos para obtener la expresión del sistema y pasarlo a forma matricial.

Este es el método de doble paso de Traub:

$$y_k = x_k - \frac{f(x_k)}{f'(x_k)}$$

$$x_{k+1} = y_k - \frac{f(y_k)}{f'(x_k)}$$

Obtenemos primero la expresión de y_k :

$$y_k = x_k - \frac{\frac{1}{x_k} - a}{\frac{-1}{x_k^2}} = x_k + \frac{(1 - ax_k)x_k^2}{x_k} = x_k + x_k - ax_k^2 = 2x_k - ax_k^2$$

Obtenemos ahora la expresión de $f(y_k)$:

$$f(y_k) = \frac{1}{x_k(2 - ax_k)} - a = \frac{1 - ax_k(2 - ax_k)}{x_k(2 - ax_k)} = \frac{1 - 2ax_k + (ax_k)^2}{x_k(2 - ax_k)} = \frac{(1 - ax_k)^2}{x_k(2 - ax_k)}$$

Obtenemos la expresión de $\frac{f(y_k)}{f'(x_k)}$:

$$\frac{f(y_k)}{f'(x_k)} = \frac{\frac{(1 - ax_k)^2}{x_k(2 - ax_k)}}{\frac{-1}{x_k^2}} = \frac{-(1 - ax_k)^2 x_k}{(2 - ax_k)}$$

Obtenemos la expresión de x_{k+1} :

$$x_{k+1} = x_k(2 - ax_k) - \frac{-(1 - ax_k)^2 x_k}{(2 - ax_k)} = \frac{x_k(2 - ax_k)^2 + (1 - ax_k)^2 x_k}{(2 - ax_k)}$$

$$= \frac{x_k(4 + a^2 x_k^2 - 4ax_k) + (1 - 2ax_k + a^2 x_k^2)x_k}{(2 - ax_k)} = \frac{4x_k + a^2 x_k^3 - 4ax_k^2 + x_k - 2ax_k^2 + a^2 x_k^3}{(2 - ax_k)}$$

$$= \frac{2a^2 x_k^3 - 6ax_k^2 + 5x_k}{(2 - ax_k)} = \frac{x_k(2a^2 x_k^2 - 6ax_k + 5)}{(2 - ax_k)}$$

Obtenemos la expresión de forma matricial de X_{k+1} a partir la hallada con los elementos:

$$X_{k+1} = (2I - AX_k)^{-1} X_k (2A^2 X_k^2 - 6AX_k + 5I)$$

Por lo tanto, se puede observar como tras haber trabajado con Traub el resultado matricial aún seguimos teniendo una expresión con inversa, que es aún más compleja que la inicial, por la que Traub no nos sirve. Es por eso que vamos a probar otro paso

5.2. Método multipaso congelando la evaluación

Este es el método que se nos propone de paso doble:

$$y_k = x_k - \frac{f(x_k)}{f'(x_k)}$$

$$x_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)}$$

Como se puede observar la expresión de y_k no ha cambiado, entonces es la misma que en el método de Traub:

$$y_k = x_k(2 - ax_k)$$

Tenemos que la expresión de $f'(y_k)$ es:

$$f'(y_k) = \frac{-1}{x_k^2(2 - ax_k)^2}$$

Obtenemos la expresión de $\frac{f(x_k)}{f'(y_k)}$:

$$\frac{f(x_k)}{f'(y_k)} = \frac{\frac{1}{x_k} - a}{\frac{-1}{x_k^2(2 - ax_k)^2}} = \frac{\frac{1 - ax_k}{x_k}}{\frac{-1}{x_k^2(2 - ax_k)^2}} = \frac{(1 - ax_k)x_k^2(2 - ax_k)^2}{-x_k} = -(1 - ax_k)x_k(2 - ax_k)^2$$

Obtenemos la expresión de x_{k+1} :

$$\begin{aligned} x_{k+1} &= x_k(2 - ax_k) - (-(1 - ax_k)x_k(2 - ax_k)^2) \\ &= x_k(2 - ax_k) + (1 - ax_k)x_k(2 - ax_k)^2 \\ &= x_k(2 - ax_k)[1 + (1 - ax_k)(2 - ax_k)] \\ &= x_k(2 - ax_k)[1 + 2 - 2ax_k - ax_k + a^2x_k^2] \\ &= x_k(2 - ax_k)(3 - 3ax_k + a^2x_k^2) \\ &= x_k(6 - 9ax_k + 5a^2x_k^2 - a^3x_k^3) \end{aligned}$$

Obtenemos la expresión de forma matricial de X_{k+1} a partir la hallada con los elementos:

$$X_{k+1} = X_k(6I - 9AX_k + 5A^2X_k^2 - A^3X_k^3)$$

Por lo tanto, se puede observar como tras haber trabajado con este nuevo paso el resultado matricial no tiene inversa, teniendo un menor coste computacional.

Análisis de Convergencia

Para determinar la convergencia del método, analizamos la evolución del error E_{k+1} siguiendo un procedimiento análogo al del Ejercicio 1. Consideramos la relación fundamental $AX_k = I - E_k$.

Partimos de la ecuación de iteración simplificada:

$$X_{k+1} = X_k (6I - AX_k (9I + AX_k (5I - AX_k)))$$

Sustituyendo en la expresión del error $E_{k+1} = I - AX_{k+1}$ y operando algebraicamente:

$$\begin{aligned} E_{k+1} &= I - AX_k (6I - AX_k (9I + AX_k (5I - AX_k))) \\ &= I - AX_k (6I - AX_k (9I + AX_k (4I + E_k))) \\ &= I - AX_k (6I - AX_k (13I - 4E_k + AX_k E_k)) \\ &= I - AX_k (6I - AX_k (13I + E_k (-4I + AX_k))) \\ &= I - AX_k (6I - AX_k (13I + E_k (-3I - E_k))) \\ &= I - AX_k (6I - 13AX_k + AX_k E_k (3I + E_k)) \\ &= I - AX_k (6E_k - 7AX_k + AX_k E_k (3I + E_k)) \\ &= I - AX_k (-7AX_k + E_k (6I + 3AX_k + AX_k E_k)) \\ &= I - AX_k (-7AX_k + E_k (9I - 3E_k + AX_k E_k)) \\ &= I - AX_k (-7AX_k + E_k (9I + E_k (-3I + AX_k))) \\ &= I - AX_k (-7AX_k + E_k (9I + E_k (-2I - E_k))) \\ &= I - AX_k (-7AX_k + 9E_k - 2E_k^2 - E_k^3) \\ &= I - AX_k (-7I + 16E_k - 2E_k^2 - E_k^3) \end{aligned}$$

Expandiendo la expresión resultante y agrupando términos en función de E_k :

$$\begin{aligned} E_{k+1} &= I + 7AX_k - 16AX_k E_k + 2AX_k E_k^2 + AX_k E_k^3 \\ &= 8I - 7E_k - 16AX_k E_k + 2AX_k E_k^2 + AX_k E_k^3 \\ &= 8I - E_k (7I + 16AX_k - 2AX_k E_k - AX_k E_k^2) \\ &= 8I - E_k (23I - 16E_k - 2AX_k E_k - AX_k E_k^2) \\ &= 8I - E_k (23I - E_k (16I + 2AX_k + AX_k E_k)) \\ &= 8I - E_k (23I - E_k (18I - 2E_k + AX_k E_k)) \\ &= 8I - E_k (23I - E_k (18I + E_k (-2I + AX_k))) \\ &= 8I - E_k (23I - E_k (18I + E_k (-I - E_k))) \\ &= 8I - 23E_k + 18E_k^2 - E_k^3 - E_k^4 \end{aligned}$$

Finalmente, factorizamos el polinomio matricial resultante utilizando las raíces numéricas halladas:

$$E_{k+1} \approx -(E_k + 5,28I)(E_k - 2,84I)(E_k^2 - 1,45E_k + 0,53I)$$

Aplicando normas a ambos lados de la ecuación y utilizando la propiedad $\|AB\| \leq \|A\| \|B\|$:

$$\begin{aligned} \|E_{k+1}\| &= \|-(E_k + 5,28I)(E_k - 2,84I)(E_k^2 - 1,45E_k + 0,53I)\| \\ &= \|(E_k + 5,28I)(E_k - 2,84I)(E_k^2 - 1,45E_k + 0,53I)\| \\ &\leq \|E_k + 5,28I\| \cdot \|E_k - 2,84I\| \cdot \|E_k^2 - 1,45E_k + 0,53I\| \end{aligned}$$

Conclusión: El análisis no muestra un resultado concluyente sobre la convergencia global. Aunque se cumpla la condición inicial $\|E_k\| < 1$, los términos independientes dentro de los factores (como 5,28 y 2,84) aumentan la cota superior del error en las primeras iteraciones, impidiendo garantizar que $\|E_{k+1}\| < \|E_k\|$.

Por otro lado hemos programado el método para obtener una idea a la hora de intentar demostrar si el método era o no convergente. En los resultado obtenidos hemos visto que el método no converge nunca. Ponemos aquí dos ejemplos, uno para $n = 10$ y otro $n = 50$, ambos con una aproximación inicial de la forma $X_0 = \beta \frac{1}{\|A\|^2}$, una $\text{tol} = 1\text{e-}10$ y un maximo de iteraciones de 10000. Los resultado obtenidos son los siguientes:

n	Iter	$\ X_{k+1} - X_k\ $	Incre	ACOC	TCC	TCL	Time
10	10000	NaN	∞	0.9999	NaN	NaN	0.09031
50	10000	NaN	∞	0.9999	NaN	NaN	3.68656

Como podemos ver no se obtienen resultados concluyentes, pues siempre se alcanza el número máximo de iteraciones y el incremento se va a infinito, además de que las tasas y la diferencia entre iterados nos dan un 'Not a Number'. Con esto y nuestro desarrollo teórico podemos conjeturar que el método no es convergente aunque no tengamos la demostración analítica.

Referencias

1.Improving Newton–Schulz Method for Approximating Matrix Generalized Inverse by Using Schemes with Memory.